Kant

Reverse Engineering (rev)

After staing like 1h to try to beat the game I got this stupid message:

```
You stand before an ancient stone tablet. Its glyphs sharpen...

      Congrats! Here is your flag:
45c6fd945ef62a0a6b20f030cf9d2e2ee2b0b781db87603ff386855086488a6045e
01993

  (...but the glyphs seem to shift every time you blink. press q.)
stty: invalid argument
'4500:5:f00bf:8a3b:3:1c:7f:15:4:0:1:0:11:13:1a:0:12:f:17:16:0:0:0:0
:0:0:0:0:0:0:0:0:0:0:0'
```

After some more inspection it looked like a rust binary. I found this:
`OMNICTF{n0t_th3_r34l_0n3_keep_d1gg1ng}`

I then found I can do `kant check <hex>`. So that was my key.

I tryed:

```
./kant check $(python3 -c
"print(open(0).read().strip().encode().hex())" <<< 'A')
```

And it told me it needs 36 bytes of data.

After a lot of digging around (and calling in a friend who knows cripto) I got the flag.

I asked my friend to make this part of the writeup and he gave me this:

```
Cipher pipeline (forward)
Input is 36 bytes (typically OMNICTF{ + 27 bytes + }).
```

```
A = input ⊕ K0
Per 4-byte block: AES SubBytes (affine 0x63) then AES MixColumns
Byte map: C[i] = ((B[i]+1)^17 mod 257 − 1) & 0xff (bijective on
GF(257)^*)
D0 = Feistel₀(C) — 16 rounds, 18+18 halves, keys of 18 bytes each,
AES S-box 0x63
SRC[i] = D0[p1[i]] (gather)
Bit rotate left by 13 over 288 bits, Rust BitVec order: bit i uses
bit index (~i) & 7 inside byte i >> 3
SB[i] = S8F[BIT[i]] — AES-style S-box with affine constant 0x8f
D1 = Feistel₁(SB) — same layout, different keys, S-box 0x8f
P11[i] = ((D1[i]+1)^11 mod 257 − 1) & 0xff
MID = P11 ⊕ K1
Z[i] = MID[p2[i]] → compare to CONST
```

To verify:

```
./kant check $(python3 -c
"print(open(0).read().strip().encode().hex())" <<<
'OMNICTF{t1m3_x0r_1s_r3v3rs1bl3_cr4p}')
```

(Solved by Bogdan and Bochis)